项目提示词

实验概要

实验名称：视觉搜索实验

假设/效应：{简述}

被试时长：{总时长，分钟}

总体结构：3 个 block 每个 block 40 个

刺激与素材（选择性填充，不填我就默认设定）

视觉刺激：{文本/图片/形状/视频；字体、字号、颜色、位置、大小、遮罩}

屏幕与硬件

窗口：背景色：白色；窗口：全屏；屏幕比率：自适用屏幕比率
显示器参数：屏幕尺寸：各种常见的笔记本电脑的尺寸
输入设备：
外设集成：

实验流程

试次结构：{刺激顺序、呈现时长、SOA/ISI、遮罩、反馈}
时间控制：{基于时钟/帧、每段精确时长}
响应收集：{允许时窗、超时策略、记录按键/RT/正确性}
随机化/平衡：{打乱方式、避免连串规则、拉丁方/完全平衡}
条件与因子：{自变量因子及水平；是否被试内/间；配比}
自适应程序（可选）：{Staircase(阈值规则、步长、停止条件)/QUEST 参数}
练习与反馈：{练习试次数、错误阈值再次练习、反馈文案与时长}
区块/休息：{block 数量、每 block trial 数、插入休息与倒计时}

数据记录

输出目录：`data/participant01` 这样类推
文件命名：`{expName}_{participant}_{date}.csv`
数据列：{必需列清单：trialN、block、cond、stimID、resp、corr、RT、onset、offset、触发码等}
时间戳：{实验开始/trial onset/刺激 onset/响应/触发的精确记录}

被试与会话

信息采集：弹窗采集：participant, age, gender 三项
分组/平衡：（木子哥填充）
本地化：中文

误差与退出

退出键：`shift + E`
容错：{素材缺失/设备不可用时的降级策略}
调试模式：{是否加入快速模式/减少trial/显示边框与坐标}

复现与依赖

PsychoPy 版本：psychopy (>=2025.2.1,<2026.0.0)

Python 版本：3.10

第三方包：不使用除了 `PsychoPy` 的其他第三方库

项目结构：`见README.md`

随机种子：按被试设定

产出物期望

需要的输出：

主程序 experiment_main.py

练习程序 run_practice.py

补完README 使用说明（运行步骤、依赖、注意事项）

其他补充说明

伪代码

```
class Trial:
    """
        The Trial Class will automatically initialize the instence attribute named grid of
the basis of is_red_matrix and t_count
    """









def main():
    trial_of_black01_01 = Trial(is_red_matrix=False, t_count=5)
    trial_of_black01_02 = Trial(is_red_matrix=False, t_count=5)
    trial_of_black01_03 = Trial(is_red_matrix=False, t_count=5)
    trial_of_black01_04 = Trial(is_red_matrix=False, t_count=5)
    trial_of_black01_05 = Trial(is_red_matrix=False, t_count=5)
    trial_of_black01_06 = Trial(is_red_matrix=True, t_count=5)
    trial_of_black01_07 = Trial(is_red_matrix=True, t_count=5)
    trial_of_black01_08 = Trial(is_red_matrix=True, t_count=5)
    trial_of_black01_09 = Trial(is_red_matrix=True, t_count=5)
    trial_of_black01_10 = Trial(is_red_matrix=True, t_count=5)
```

```
trial_of_black01_11 = Trial(is_red_matrix=False, t_count=10)
trial_of_black01_12 = Trial(is_red_matrix=False, t_count=10)
trial_of_black01_13 = Trial(is_red_matrix=False, t_count=10)
trial_of_black01_14 = Trial(is_red_matrix=False, t_count=10)
trial_of_black01_15 = Trial(is_red_matrix=False, t_count=10)
trial_of_black01_16 = Trial(is_red_matrix=True, t_count=10)
trial_of_black01_17 = Trial(is_red_matrix=True, t_count=10)
trial_of_black01_18 = Trial(is_red_matrix=True, t_count=10)
trial_of_black01_19 = Trial(is_red_matrix=True, t_count=10)
trial_of_black01_20 = Trial(is_red_matrix=True, t_count=10)

trial_of_black01_21 = Trial(is_red_matrix=False, t_count=15)
trial_of_black01_22 = Trial(is_red_matrix=False, t_count=15)
trial_of_black01_23 = Trial(is_red_matrix=False, t_count=15)
trial_of_black01_24 = Trial(is_red_matrix=False, t_count=15)
trial_of_black01_25 = Trial(is_red_matrix=False, t_count=15)
trial_of_black01_26 = Trial(is_red_matrix=True, t_count=15)
trial_of_black01_27 = Trial(is_red_matrix=True, t_count=15)
trial_of_black01_28 = Trial(is_red_matrix=True, t_count=15)
trial_of_black01_29 = Trial(is_red_matrix=True, t_count=15)
trial_of_black01_30 = Trial(is_red_matrix=True, t_count=15)

trial_of_black01_31 = Trial(is_red_matrix=False, t_count=20)
trial_of_black01_32 = Trial(is_red_matrix=False, t_count=20)
trial_of_black01_33 = Trial(is_red_matrix=False, t_count=20)
trial_of_black01_34 = Trial(is_red_matrix=False, t_count=20)
trial_of_black01_35 = Trial(is_red_matrix=False, t_count=20)
trial_of_black01_36 = Trial(is_red_matrix=True, t_count=20)
trial_of_black01_37 = Trial(is_red_matrix=True, t_count=20)
trial_of_black01_38 = Trial(is_red_matrix=True, t_count=20)
trial_of_black01_39 = Trial(is_red_matrix=True, t_count=20)
trial_of_black01_40 = Trial(is_red_matrix=True, t_count=20)

# 重复上述步骤，生成 trial_of_black02_01, ... trial_of_black02_40
# 重复上述步骤，生成 trial_of_black03_01, ... trial_of_black03_40




black01 = random.shuffle([trial_of_black01_01, ... trial_of_black01_40]) # 乱序

black02 = random.shuffle([trial_of_black02_01, ... trial_of_black02_40]) # 乱序

balck03 = random.shuffle([trial_of_black03_01, ... trial_of_black03_40]) # 乱序

# 根据三个 balck 的序列，呈现 trial
```

- **Trial Class**：
  - `is_red_matrix`：布尔值，表示当前试次是否会出现红色大T。
  - `grid`：一个5x5的整数矩阵，表示网格中每个位置的状态。数值含义如下：

        - 0：空位置
        - 1：正立的蓝色大T
        - 2：正立的红色大T
        - 3：倒立的蓝色大T
        - 4：倒立的红色大T
    - `t_count`：整数，表示试次中蓝色大T的数量。
- **方法**：
    - `generate_blue_t()`：根据`t_count`的值，在`grid`中随机生成相应数量的蓝色大T（随机分配正立或倒立状态，即值为1或3）。
    - `ensure_red_t()`：如果`is_red_matrix`为True，随机选择一个蓝色大T（值为1或3），并将其替换为对应的红色大T（正立换为正立红色，倒立换为倒立红色）。